Jeffrey James Research Doc for SVG to GCode Pipleline

Questions to Answer:

- What can be done to reduce the time of the drawing
    - Reduce Number of Instructions
    - Makes motion paths more continuous rather than being segmented
    - Increase speed that arduino processes the data
    - Increase speed of motors
    - Get Rid of double edges (this is more under Sara's domain)
- What are the tradeoffs with potentially increasing speed?
    - Increased speed will most likely lead to decreased accuracy
    - More Jerkly and faster movement may put strain on the wires and motors
    - Will data get lost if we are transferring instructions too quickly
- How should the pipeline handle color?

Research:
Approach 1: CMYK Color Separation (Traditional Print Method)

Split the image into 4 layers (Cyan, Magenta, Yellow, Black), each drawn with a different pen.

Algorithm (from https://gist.github.com/llwyd/6051c0d1b78ecf2e6954bde7ac09d63c):

```
from PIL import Image
from PIL import ImageCms

def separate_cmyk(input_path):
    rgb_icc = ImageCms.createProfile('sRGB')
    cmyk_icc = ImageCms.getOpenProfile('path/to/cmyk.icc')

    im = Image.open(input_path).convert('RGB')
    im_cmyk = ImageCms.profileToProfile(im, rgb_icc, cmyk_icc, outputMode='CMYK')

    c, m, y, k = im_cmyk.split()
    return {'cyan': c, 'magenta': m, 'yellow': y, 'black': k}
```

Each channel becomes a grayscale image representing ink density for that color.

Approach 2: Hatching with vpype Plugins

Convert grayscale intensity to line density using https://github.com/plottertools/hatched:
pip install hatched
vpype hatched --levels 64 128 192 -s 0.5 -p 4 input.jpg write output.svg
For crosshatching with color, use https://github.com/serycjon/vpype-flow-imager:
pip install vpype-flow-imager
Crosshatch with two passes at 90° rotation
vpype flow_img -fs 42 image.jpg flow_img -fs 42 --rotate 90 image.jpg write output.svg

Approach 3: Color Quantization to Available Pens

Match image colors to your available pen colors (you have: Black, Red, Green, Blue, Purple based on color_widget.py):

```
from PIL import Image
import numpy as np

def quantize_to_pens(image_path, pen_colors):
    """Reduce image colors to match available pen colors"""
    img = Image.open(image_path).convert('RGB')
    # Use PIL's quantize or custom nearest-neighbor matching
    palette_img = img.quantize(colors=len(pen_colors), method=Image.MEDIANCUT)
    return palette_img
```

-----Implementation: Multi-Layer G-code with Pen Changes

The key is configuring vpype-gcode to pause between layers. Update your `config.toml`:

```
[gwrite]
default_profile = "grbl"

[gwrite.grbl]
unit = "inch"
document_start = "G20\nG90\nG92 X0 Y0\nM3 S70\n"

# Pause and prompt for pen change at each layer
layer_start = "M3 S70\nM0 (Change to {vp_name} pen)\n"

segment_first = "M3 S70\nG00 X{x:.4f} Y{y:.4f}\nM3 S250\n"
segment = "G01 X{x:.4f} Y{y:.4f}\n"
layer_end = "M3 S70\n"
document_end = "M3 S70\nM5\nM2\n"
```

The `M0` command pauses execution until the user resumes (after changing pens). `{vp_name}` inserts the layer name.-----Recommended Implementation Path

Step	What	Tools

1	Color separation	PIL/Pillow CMYK split or color quantization
2	Raster → Vector	hatched plugin (converts grayscale to hatched lines)
3	Layer organization	vpype `--attr stroke` to preserve colors as layers
4	Pen change pauses	`layer_start = "M0 (Change pen)..."` in `config.toml`
5	G-code generation	vpype-gcode with updated config

-----Code Integration Options

Option A: Full Pipeline (Recommended)

Image → CMYK Split → Hatching per channel → vpype layers → G-code with M0 pauses

Option B: Simpler - Color Quantization

Image → Quantize to 5 pen colors → Separate SVG layers → G-code with M0 pauses

Option C: Use Existing Tools

- https://docs.drawingbotv3.com/en/latest/pfms.html has built-in CMYK separation and

hatching
- https://github.com/buildmase/cursor-pen-plotter has crosshatch scripts

-----Sources

- https://github.com/abey79/vpype
- https://vpype.readthedocs.io/en/stable/cookbook.html
- https://github.com/plottertools/vpype-gcode
- https://github.com/plottertools/hatched
- https://github.com/serycjon/vpype-flow-imager
- https://code.adonline.id.au/cmyk-in-python/
- https://hackaday.io/project/4823-color-crosshatching-with-pen-plotter
- https://geoffinthegarage.net/blog/optimized-pen-plotting/

Follow Up Question:

Why use CMYK rather than RGB?

RGB is for things that emit light (screens). CMYK is for things that reflect light (paper + ink).

Ink does NOT emit light — it **absorbs light** and reflects what's left into your eyes.**How colors form:**

- Cyan absorbs red
- Magenta absorbs green
- Yellow absorbs blue

When light hits paper:

- More ink = more light absorbed = darker color
- No ink = white paper

This is called **subtractive mixing** because ink removes wavelengths of light

Pixels **emit light**.

Each pixel has red, green, and blue subpixels that glow at different intensities.**How colors form:**

- Red + Green = Yellow
- Red + Blue = Magenta
- Green + Blue = Cyan
- Red + Green + Blue = **White**
- No light = **Black**

This is called **additive mixing** because you *add light* to get brighter colors.

Vpype Color Quanitization Properties:

vpype Multi-Color Capabilities
vpype has robust built-in support for multi-color workflows. Here's a comprehensive overview:

Core Concept: Layers = Colors
In vpype, each layer represents a different pen/color. Layers are identified by positive integers (1, 2, 3...) and each has properties:
- vp_color - the color
- vp_pen_width - line thickness
- vp_name - layer name (e.g., "cyan", "black")

Importing Colors from SVG

Default behavior: Groups SVG by top-level groups (like Inkscape layers)

Sort by stroke color using --attr:
Create one layer per unique color in the SVG
vpype read --attr stroke input.svg write output.svg

Sort by both color AND stroke width
vpype read --attr stroke --attr stroke-width input.svg write output.svg

This automatically separates a multi-colored SVG into layers.

Command	Purpose	Example
color	Set layer color	`color --layer 2 #ff0000`
penwidth	Set pen width	`penwidth --layer 1`

		`0.5mm`
name	Name a layer	`name --layer 1 "black"`
pens	Apply preset config	`pens cmyk`
lmove	Move content between layers	`lmove 1,2 1`
lcopy	Copy content to another layer	`lcopy 1 new`
ldelete	Delete a layer	`ldelete 3`

Built-in Pen Configurations

vpype includes preset pen configs, like CMYK:

vpype read input.svg pens cmyk write output.svg

This assigns:
- Layer 1 → Cyan
- Layer 2 → Magenta
- Layer 3 → Yellow
- Layer 4 → Black (Key)

Processing Each Layer Separately

Use forlayer to run commands per-layer or export separate files:

Export each color as a separate file
vpype read input.svg forlayer write "output_{_name}.svg" end

Apply different processing per layer
vpype read input.svg forlayer linesimplify -t 0.5mm end write output.svg

Available variables inside `forlayer`:

- `{_lid}`: layer ID (1, 2, 3...)
- `{_name}`: layer name
- `{_color}`: layer color
- `{_i}`: iteration index
- `{_n}`: total layer count

G-code with Pen Changes

In your config.toml, use layer_start to pause between colors:

```
[gwrite.grbl]
# Pause and display pen name at each layer change
layer_start = "M3 S70\nG00 X0 Y0\nM0 (Change to {vp_name} pen)\n"
```

The {vp_name} variable inserts the layer name (e.g., "cyan", "black").

Complete Multi-Color Workflow Example

```
# 1. Read SVG, separate by color
# 2. Apply CMYK pen names
# 3. Optimize each layer
# 4. Export to G-code with pen change pauses

vpype \
  read --attr stroke input.svg \
  pens cmyk \
  linemerge linesimplify reloop linesort \
  gwrite --config config.toml output.gcode
```

What vpype Does NOT Do

vpype handles vector color separation, but does not:
- Convert raster images to vectors (use hatched plugin or potrace)
- Perform CMYK color separation of photographs (use PIL/Pillow first)
- Generate halftone/crosshatch patterns (use hatched or vpype-flow-imager plugins)

For raster → multi-color vectors, the pipeline is:
Image → CMYK split (PIL) → Hatching per channel (hatched) → vpype layers → G-code

---
Sources

- https://vpype.readthedocs.io/en/stable/fundamentals.html
- https://vpype.readthedocs.io/en/latest/reference.html
- https://vpype.readthedocs.io/en/latest/cookbook.html
- https://github.com/abey79/vpype

- https://bylr.info/articles/2022/03/03/annotated-release-notes-vpype-1.9/

Motor Speed:

The F parameter sets speed in units per minute (inches/min or mm/min based on G20/G21).

Update config.toml to include feed rates:

```
[gwrite]
default_profile = "grbl"

[gwrite.grbl]
unit = "inch"
document_start = "G20\nG90\nG92 X0 Y0\nM3 S70\n"
layer_start = ""

# Travel move (pen up) - G00 uses max speed, no F needed
segment_first = "M3 S70\nG00 X{x:.4f} Y{y:.4f}\nM3 S250\n"

# Drawing move (pen down) - F sets speed in inches/min
segment = "G01 X{x:.4f} Y{y:.4f} F300\n"

layer_end = ""
document_end = "M3 S70\nM5\nM2\n"
info = "GRBL profile with feed rate"
```

Common Feed Rates

Speed	inches/min	mm/min	Use Case
Slow	60-120	1500-3000	Fine detail
Medium	200-400	5000-10000	Normal drawing
Fast	500-800	12000-20000	Quick sketches

Other Ways to Control Speed

In firmware (GRBL settings):

$110 = 5000 ; X max rate (mm/min)
$111 = 5000 ; Y max rate (mm/min)
$120 = 500 ; X acceleration (mm/sec²)

1. $121 = 500   ; Y acceleration (mm/sec²)
2. **Manual G-code command (via your manual send box):**
   G01 F600   ; Set feed rate to 600 units/min (persists for subsequent moves)

3. Once per file - Set F once in document_start, it persists:
document_start = "G20\nG90\nG92 X0 Y0\nG01 F300\nM3 S70\n"

3. "
   Then `segment` doesn't need F repeated every line (reduces file size).